

Question Index - All 110 Questions

Q#	Question
Q1	What is CI/CD? Continuous Integration vs Delivery vs Deployment
Q2	CI/CD pipeline anatomy - stages, jobs, steps, triggers
Q3	Pipeline as Code - why YAML/Groovy beats clicking a UI
Q4	Build triggers - push, PR, schedule, tag, manual, webhook
Q5	Environment variables and secrets in CI/CD
Q6	Artifacts vs caching - passing data between stages
Q7	Branch strategies - trunk-based vs GitFlow vs GitHub Flow
Q8	Jenkins architecture - controller, agents, plugins
Q9	Jenkinsfile deep dive - declarative vs scripted syntax
Q10	Jenkins Multibranch Pipeline - auto-discovering branches and PRs
Q11	Jenkins credentials management - stores, scoping, rotation
Q12	Jenkins Blue Ocean - modern UI and pipeline visualization
Q13	GitHub Actions architecture - workflows, jobs, steps, runners
Q14	GitHub Actions YAML deep dive - conditionals, matrix, expressions
Q15	GitHub Actions environments, secrets scoping, and protection rules
Q16	GitHub Actions marketplace - using and creating custom actions
Q17	GitLab CI architecture - .gitlab-ci.yml, stages, runners, executors
Q18	GitLab CI YAML deep dive - includes, extends, rules, needs, DAG
Q19	GitLab CI environments, deployment tracking, and Auto DevOps
Q20	GitLab CI merge trains - automated merge queues
Q21	Docker in CI - building and pushing images in pipelines
Q22	Basic testing in pipelines - unit, integration, smoke tests

Q23	Pull request pipelines and preview environments
Q24	Build agents/runners - self-hosted vs cloud-hosted
Q25	Notifications, dashboards, and pipeline reporting
Q26	Concurrency control - preventing parallel deployments
Q27	Rollback strategies in CI/CD pipelines
Q28	Jenkins vs GitHub Actions vs GitLab CI - when to use which
Q29	Jenkins shared libraries - DRY pipeline code across 50+ teams
Q30	Jenkinsfile advanced - post blocks, when conditions, parameters, input
Q31	GitHub Actions reusable workflows and composite actions
Q32	GitHub Actions Docker actions vs JavaScript actions vs composite
Q33	GitHub Actions self-hosted runner groups, labels, and routing
Q34	GitLab CI parent-child pipelines and multi-project pipelines
Q35	Pipeline security - supply chain attacks and defenses
Q36	Secrets management - Vault, OIDC, short-lived credentials
Q37	Webhook security - signatures, verification, preventing spoofing
Q38	Container image building in CI - Kaniko, BuildKit, buildx
Q39	Infrastructure pipeline - Terraform plan/apply in CI/CD
Q40	Database migrations in CI/CD - safe schema changes
Q41	End-to-end testing - Cypress, Playwright, Selenium in pipelines
Q42	Code quality gates - SonarQube, coverage thresholds, linting
Q43	SAST, DAST, and license scanning in CI/CD
Q44	Deployment strategies - blue-green, canary, rolling from CI/CD
Q45	GitOps integration - CI builds, CD deploys via ArgoCD/Flux
Q46	ArgoCD Image Updater - automatic image tag updates

Q47	Feature flags and CI/CD - decoupling deployment from release
Q48	Pipeline caching strategies - reducing build times by 80%
Q49	Parallelism and matrix builds - testing across versions/platforms
Q50	Approval gates and deployment windows
Q51	Pipeline templates and standardization across teams
Q52	Test result reporting - JUnit XML, flaky test detection
Q53	Ephemeral environments - preview environments per PR
Q54	Versioning and tagging - semver, calver, semantic-release
Q55	GitHub Actions security - permissions, GITHUB_TOKEN, OIDC
Q56	Monorepo CI - detecting changes and building only what changed
Q57	Pipeline timeout, retry strategies, and idempotency
Q58	Build reproducibility - hermetic builds, deterministic outputs
Q59	Container registry integration - ECR, GCR, Harbor from CI
Q60	Helm and Kubernetes deployments from CI/CD
Q61	Multi-environment promotion - dev to staging to prod
Q62	Pipeline cost optimization - reducing CI/CD spend
Q63	CI/CD architecture at scale - 500+ developers, 100+ services
Q64	Jenkins at scale - HA, shared library governance, plugin management
Q65	Pipeline observability - metrics, traces, DORA metrics
Q66	Compliance and audit - SOC2, HIPAA, change management in CI/CD
Q67	Pipeline security hardening - SLSA levels, Sigstore, provenance
Q68	Multi-cloud CI/CD - deploying to AWS, GCP, Azure from one pipeline
Q69	Migrating from Jenkins to GitHub Actions or GitLab CI
Q70	Advanced rollback - database-aware, feature-flag, traffic-based

Q71	CI/CD for microservices - contracts, dependency ordering
Q72	Trunk-based development at scale - short branches, feature flags
Q73	Tekton Pipelines - Kubernetes-native CI/CD
Q74	Pipeline as a product - internal developer platforms, Backstage
Q75	Disaster recovery for CI/CD infrastructure
Q76	Progressive delivery - Argo Rollouts, Flagger from CI/CD
Q77	Release management - semver, changelogs, release trains
Q78	CI/CD for mobile apps - iOS, Android pipelines
Q79	Zero-downtime deployment verification - smoke tests, synthetic monitoring
Q80	Cross-tool pipeline triggers - webhook chaining across systems
Q81	CircleCI, Azure DevOps, Bitbucket Pipelines - alternative tools
Q82	AI in CI/CD - test selection, flaky detection, auto-remediation
Q83	Change freeze and deployment windows - governance automation
Q84	Pipeline as DAG vs linear stages - modern execution models
Q85	Pipeline takes 45 minutes - optimize to under 10
Q86	Production deployment broke - rollback with database changes
Q87	Secret leaked in CI logs - incident response
Q88	CI runner compromised - supply chain attack response
Q89	Flaky test blocking all deployments for a week
Q90	Two teams deployed conflicting changes simultaneously
Q91	Jenkins controller disk full - builds failing cluster-wide
Q92	GitHub Actions fork stealing secrets via pull_request_target
Q93	Deployment succeeded but app is broken - no rollback plan
Q94	Pipeline costs tripled this month - audit and optimize

Q95	Helm upgrade in CI failed midway - release stuck in pending
Q96	Feature branch merged without passing CI - broken main
Q97	Docker image wrong architecture - crashes in prod
Q98	Terraform apply in CI destroyed production resources
Q99	CI pipeline leaking credentials through build artifacts
Q100	Coordinated deployment to 50 microservices
Q101	Migrating 200 Jenkins jobs to GitHub Actions
Q102	Jenkins shared library update broke 50 team pipelines
Q103	workflow_dispatch with wrong inputs deployed wrong branch to prod
Q104	GitLab CI runner token leaked - unauthorized pipelines running
Q105	Pipeline deploying stale artifact - cache returning old build
Q106	CI/CD platform outage - can't deploy critical hotfix
Q107	Dependency update broke production - no lockfile pinning
Q108	Pipeline YAML merge conflict silently removed a stage
Q109	Webhook flood - 1000 builds triggered by bot commits
Q110	Rollback deployed but old image garbage-collected from registry

SECTION 1 - Junior Core Concepts (Q1-Q28)

Q1 What is CI/CD? Continuous Integration vs Delivery vs Deployment

The Simple Answer

CI/CD automates the path from code commit to production. Continuous Integration means developers merge frequently and every merge triggers automated tests. Continuous Delivery means code is always in a deployable state. Continuous Deployment means every passing change goes to production automatically.

The Real-World Analogy

Think of a car manufacturing plant. CI is the quality inspection station - every part is tested as it arrives, catching defects immediately. Continuous Delivery means the finished car is parked in the shipping bay, ready whenever the customer says ship it. Continuous Deployment means the car drives itself to the customer the moment it passes inspection. Most companies practice Delivery - the car is ready, but a human makes the final call.

The Critical Distinction

Most companies say CI/CD but do CI with manual deployments. True CI requires: every developer merges to main at least daily, every merge triggers automated tests, and broken builds are fixed within minutes. If your team has branches lasting weeks, that is not CI regardless of what tools you use.

CI/CD Is a Practice, Not a Tool

Jenkins, GitHub Actions, and GitLab CI are tools. CI/CD itself is a development practice. You can have Jenkins running pipelines and still not do CI if developers do not merge frequently or fix broken builds quickly. The tools enable the practice but do not guarantee it.

Q2 CI/CD pipeline anatomy - stages, jobs, steps, triggers

The Simple Answer

A pipeline is a sequence of automated stages that code passes through from commit to production. Each stage contains jobs. Each job contains steps. Triggers define what starts the pipeline.

The Standard Pipeline

Build - Compile, install dependencies, create artifacts. Catches syntax errors. **Test** - Unit tests, integration tests, linting. Catches logic errors. **Security Scan** - SAST, dependency scanning. Catches vulnerabilities. **Package** - Build Docker image, push to registry. Creates the deployable artifact. **Deploy Staging** - Deploy to staging for verification. **Deploy Production** - Deploy to production, usually with approval.

Triggers Explained

Push - Runs on every commit. Used for fast CI feedback. **Pull request** - Runs when a PR is opened or updated. Quality gate before merge. **Tag** - Runs on tag creation. Used for releases. **Schedule** - Cron-based. Used for nightly security scans. **Manual** - Human-triggered. Used for production deployments requiring approval. **Webhook** - Triggered by an external system. Used for cross-tool integration.

Q3 Pipeline as Code - why YAML/Groovy beats clicking a UI

The Simple Answer

Pipeline as Code means defining your CI/CD pipeline in a file committed to your repository. Changes go through code review. Every pipeline change has version history.

Why It Matters

Version control - Every change has a commit, author, and timestamp. You can git blame a pipeline. **Review process** - Pipeline changes go through PRs. Bad configurations are caught in review. **Reproducibility** - The pipeline definition is tied to the code version. Checking out a historical commit includes the pipeline that built it. **Disaster recovery** - If the CI server dies, pipelines are in Git. No manual reconfiguration.

The anti-pattern: Configuring pipelines through a web UI. Changes are untraceable, unreviewable, and lost if the server fails. Every major CI tool now supports pipeline as code. There is zero reason to configure pipelines through a UI in 2025.

Q4 Build triggers - push, PR, schedule, tag, manual, webhook

The Recommended Trigger Strategy

On push to feature branches - Run lint, unit tests, and build. Fast feedback. Skip slow integration tests. **On pull request** - Full test suite, security scans, quality checks. This is the quality gate. **On merge to main** - Complete pipeline including artifact building and staging deployment. **On tag creation** - Build release artifact, trigger production deployment with approval. **On schedule (nightly)** - Full security scans, dependency audits, integration suites too slow for every commit.

Never trigger full pipelines on every push to every branch. It wastes compute, costs money, and trains developers to ignore results. Run fast checks on push, full checks on PR.

Q5 Environment variables and secrets in CI/CD

The Simple Answer

Environment variables configure pipeline behavior - branch name, build number, target environment. Secrets are sensitive values - API keys, passwords, credentials - that must be protected from exposure.

Secret Best Practices

Secrets are stored in the CI platform's secret store (GitHub Secrets, GitLab CI Variables, Jenkins Credentials). They are injected at runtime, masked in logs automatically, and scoped as narrowly as possible. Never print environment variables for debugging - this accidentally exposes secrets. Never pass secrets through build arguments that persist in Docker layers. Never commit .env files to Git.

The golden rule: Secrets exist in exactly one place - the CI platform's secret store. They never appear in code, logs, artifacts, or Docker images.

Q6 Artifacts vs caching - passing data between stages

The Simple Answer

Artifacts - Job outputs passed downstream. A build job produces a binary that a deploy job uses. Stored permanently (or with TTL). Downloadable by users.

Cache - Reused between pipeline runs to speed up builds. Dependencies cached so npm install only downloads new packages. Best-effort - a cache miss means a slower build, not a broken build.

The Key Difference

Artifacts are critical to pipeline correctness - without them, downstream jobs fail. Cache is an optimization - without it, builds are slower but still work. Design pipelines so they work without cache and use cache purely for speed.

Q7 Branch strategies - trunk-based vs GitFlow vs GitHub Flow

The Simple Answer

Trunk-based - Everyone commits to main. Branches live less than a day. Feature flags for incomplete work. Best for teams practicing true CI. Google, Facebook, Microsoft use this.

GitHub Flow - Main branch + short-lived feature branches. PRs merge to main. Simple and effective for most teams.

GitFlow - Main, develop, feature, release, hotfix branches. Each with different CI rules. Designed for versioned releases. Declining in popularity because its complexity slows delivery.

The industry trend is trunk-based development. GitFlow is declining because its complexity rarely provides proportional benefits for web applications.

Q8 Jenkins architecture - controller, agents, plugins

The Simple Answer

Jenkins uses a controller-agent architecture. The controller manages configuration, schedules jobs, and serves the UI. Agents execute build work. Plugins extend functionality - 1800+ plugins covering Git, Docker, Slack, Kubernetes, and everything else.

The Analogy - A Construction Company

The Jenkins controller is the main office - takes orders, assigns work, tracks progress. Agents are construction crews sent to job sites. Each crew has specific tools. The Jenkinsfile is the blueprint. Plugins are specialized tools added to the toolbox.

The Plugin Problem

Jenkins' greatest strength is its greatest weakness. Plugins provide flexibility but create dependency nightmares. Plugin updates can break builds. Plugin conflicts cause mysterious failures. Security vulnerabilities in plugins are common. Best practice: minimize plugins, use only well-maintained ones, and pin versions.

Controller Recommendations

Never run builds on the controller - use agents for all build work. The controller should only schedule, coordinate, and serve the UI. Running builds on the controller is a security risk (builds have access to controller filesystem) and a reliability risk (a runaway build can crash the controller, affecting all teams).

Q9 Jenkinsfile deep dive - declarative vs scripted syntax

Declarative Pipeline

The recommended approach. Structured, opinionated syntax with pipeline, stages, stage, steps blocks. Easier to read, maintain, and enforce standards. Has built-in error handling, post-build actions, and environment management.

Key Sections

pipeline - Top-level wrapper. **agent** - Where to run (any, docker, kubernetes, label). **stages** - Contains all stage definitions. **stage** - A logical group of steps (Build, Test, Deploy). **steps** - The actual commands (sh, echo, script). **post** - Actions after all stages: always (cleanup), success (notify), failure (alert), unstable (handle). **environment** - Variables available throughout or within a stage. **when** - Conditional execution: when { branch 'main' }, when

`{ changeset '/*.js' }, when { expression { return params.DEPLOY } }, parameters**` - Build parameters users can set at trigger time: string, choice, booleanParam.

Scripted Pipeline

Full Groovy programming language. Maximum flexibility. Used when declarative cannot express complex logic. Common pattern: use declarative with script blocks for the few places that need Groovy flexibility.

The interview-winning detail: Know that declarative pipelines generate a Groovy AST (Abstract Syntax Tree) internally - they are compiled to scripted pipeline under the hood. This is why you can embed script `{ }` blocks inside declarative pipelines. Understanding this architecture shows depth.

Q10 Jenkins Multibranch Pipeline - auto-discovering branches and PRs

The Simple Answer

A Multibranch Pipeline automatically discovers branches and pull requests in your SCM repository that contain a Jenkinsfile. It creates a sub-job for each branch/PR and runs the pipeline defined in the Jenkinsfile. When a branch is deleted, the corresponding job is automatically removed.

How It Works

You configure a Multibranch Pipeline with your repository URL and credentials. Jenkins scans the repository periodically (or via webhook). For each branch with a Jenkinsfile, it creates a pipeline job. The Jenkinsfile in each branch defines that branch's pipeline - feature branches might skip deployment, the main branch deploys to staging, release branches deploy to production.

Why It Matters

Without Multibranch, you manually create a Jenkins job for each branch. With Multibranch, Jenkins auto-discovers everything. This is essential for PR-based workflows where feature branches come and go frequently. Combined with GitHub or GitLab webhooks, Multibranch Pipelines provide the same PR-check experience that GitHub Actions and GitLab CI offer natively.

Branch Source Configuration

You can filter which branches get pipelines using regex patterns, exclude branches by name, and configure different behaviors for branches vs PRs. PRs typically get a different pipeline that includes checks but skips deployment.

Q11 Jenkins credentials management - stores, scoping, rotation

The Simple Answer

Jenkins stores secrets in its Credentials system. Credentials can be scoped to global (available everywhere), system (available to Jenkins internals only), or folder/job (available only within that scope). Types include username/password, SSH key, secret text, certificate, and file.

How Credentials Are Used in Pipelines

The `withCredentials` block injects credentials as environment variables for the duration of the block. After the block, the variables are unset. Jenkins masks credential values in build logs automatically - if a password appears in console output, Jenkins replaces it with asterisks.

Security Concerns

Credentials are encrypted at rest using Jenkins' master key. If someone gains access to the Jenkins controller's filesystem, they can decrypt all credentials using the master key file. This is why Jenkins controllers should have restricted access, regular backups, and the master key should be protected with filesystem permissions.

Credential Rotation

Jenkins does not rotate credentials automatically. You must manually update credential values when rotating. For cloud credentials, use short-lived tokens (IAM roles, OIDC) instead of long-lived access keys stored in Jenkins.

Best practice: Use the Jenkins credential system for all secrets - never hardcode values in Jenkinsfiles. Scope credentials to the narrowest level possible. Use OIDC or IAM roles for cloud access instead of stored access keys. Audit credential access regularly.

Q12 Jenkins Blue Ocean - modern UI and pipeline visualization

The Simple Answer

Blue Ocean is Jenkins' modern UI that provides a visual pipeline editor, better pipeline visualization with parallel stages shown graphically, and a streamlined PR/branch view. It makes Jenkins look and feel more like modern CI platforms.

Current Status

Blue Ocean's development has slowed significantly. Jenkins is focusing on the 'new look' UI updates to the classic interface instead. For new installations, many teams skip Blue Ocean entirely and use the classic UI with the modern theme, or they interact with Jenkins primarily through Jenkinsfiles and API rather than the UI.

For interviews: Know Blue Ocean exists and what it offered. But also know it is not actively developed and the industry direction is either modern Jenkins UI or migration away from Jenkins entirely. This shows realistic awareness rather than textbook knowledge.

Q13 GitHub Actions architecture - workflows, jobs, steps, runners

The Simple Answer

GitHub Actions is GitHub's native CI/CD platform. Workflows are YAML files in `.github/workflows/`. Each workflow has jobs that run in parallel by default. Each job runs on a runner and contains sequential steps.

Key Concepts in Detail

Workflow - A YAML file triggered by GitHub events. One repo can have many workflows. Workflows can call other workflows (reusable workflows).

Job - A set of steps on one runner. Jobs run in parallel unless you specify dependencies with `needs:`. Each job gets a fresh runner - no state shared between jobs unless you use artifacts.

Step - Either a shell command (`run:`) or a pre-built action (`uses:`). Steps within a job run sequentially and share the runner's filesystem.

Runner - The machine executing the job. GitHub provides hosted runners (Ubuntu, macOS, Windows) with pre-installed tools. Self-hosted runners run on your infrastructure for more control or specialized needs.

Actions - Reusable units from the marketplace. `actions/checkout`, `docker/build-push-action`, `aws-actions/configure-aws-credentials`. You can also write custom actions.

Why Teams Choose GitHub Actions

Deep GitHub integration - PR checks, issue comments, release creation all native. No separate CI server. Free for public repos. YAML is more readable than Groovy. The marketplace reduces boilerplate. OIDC support for cloud authentication eliminates stored credentials.

Q14 GitHub Actions YAML deep dive - conditionals, matrix, expressions

The Simple Answer

GitHub Actions YAML supports powerful expressions for conditional execution, matrix strategies for multi-dimensional testing, and reusable patterns for DRY workflows.

Conditionals with if:

Every job and step can have an `if:` condition. Common patterns: `if: github.ref == 'refs/heads/main'` (only on main branch), `if: github.event_name == 'pull_request'` (only on PRs), `if: contains(github.event.head_commit.message, '[skip ci])'` (skip CI on demand), `if: always()` (run even if previous steps failed - useful for cleanup).

Matrix Builds

The `strategy.matrix` field generates multiple job instances from parameter combinations. Test across Node 18, 20, 22 on Ubuntu and macOS - 6 combinations from one job definition. Use `include:` to add specific combinations and `exclude:` to remove them.

Expressions

GitHub Actions expressions use `${{ }}` syntax. Access context variables: `github.sha`, `github.ref`, `github.actor`, `runner.os`, `env.MY_VAR`. Use functions: `contains()`, `startsWith()`, `format()`, `toJSON()`, `hashFiles()`. The `hashFiles()` function is critical for caching - it generates a hash from file contents for cache keys.

Q15 GitHub Actions environments, secrets scoping, and protection rules

The Simple Answer

Environments define deployment targets (staging, production) with their own secrets, protection rules, and deployment history. This creates a governance layer - production deployments require approval, staging does not.

Secret Scoping Levels

Organization secrets - Available to all or selected repositories. Use for shared cloud credentials.

Repository secrets - Available to all workflows in the repo. Use for repo-specific API keys.

Environment secrets - Available only to jobs targeting that environment. Use for environment-specific values like production database URLs.

Protection Rules

Environments can require: manual approval from specific reviewers, wait timer (delay between approval and deployment), branch restrictions (only main can deploy to production), and custom deployment protection rules via API.

The key security pattern: Production environment requires two reviewers and only accepts deployments from the main branch. This prevents: deploying untested code (branch restriction), deploying without oversight (reviewer requirement), and accessing production secrets from feature branches (environment scoping).

Q16 GitHub Actions marketplace - using and creating custom actions

The Simple Answer

The marketplace provides pre-built actions for common tasks. Three types of custom actions exist: JavaScript (runs in Node.js, fast startup), Docker (runs in a container, any language, slower startup), and Composite (combines multiple steps into one reusable action).

Using Marketplace Actions Safely

Always pin actions by SHA, not tag: uses: actions/checkout@abc123def instead of uses: actions/checkout@v4. Tags can be moved to point to different code - a compromised tag could inject malicious code into your pipeline. SHAs are immutable. Use Dependabot or Renovate to automatically update pinned SHAs when new versions are released.

Creating Custom Actions

Composite actions - Sequence of steps in an action.yml. No additional language needed. Best for combining common step patterns.

Docker actions - Run in a container. Can use any language. Best for complex logic that needs specific dependencies.

JavaScript actions - Run directly on the runner. Fastest startup. Best for GitHub API interactions and lightweight tasks.

Q17 GitLab CI architecture - .gitlab-ci.yml, stages, runners, executors

The Simple Answer

GitLab CI is built into GitLab. Pipelines are defined in .gitlab-ci.yml at the repo root. Jobs are assigned to stages. Stages run sequentially, jobs within a stage run in parallel. Runners execute jobs using different executors.

Executors Explained

Docker executor - Each job runs in a fresh Docker container. Best isolation. Most common choice.

Kubernetes executor - Each job runs as a Kubernetes Pod. Scales automatically. Best for teams with existing Kubernetes clusters.

Shell executor - Jobs run directly on the runner machine's shell. Simplest but least isolated. Only use for trusted projects.

SSH executor - Jobs connect to a remote machine via SSH. Used for deploying to specific servers.

Runners in Detail

Shared runners - Provided by GitLab.com or shared across projects on self-managed instances. Convenient but limited in configuration.

Group runners - Available to all projects within a GitLab group. Good for team-level customization.

Project runners - Dedicated to a single project. Most control. Required for specialized hardware or security-sensitive builds.

Key Differentiators

GitLab CI has native features that require plugins or third-party tools elsewhere: built-in container registry, built-in security scanning (SAST, DAST, dependency scanning), environment tracking and deployment boards, merge request pipelines with visual diffs, and Auto DevOps for zero-config CI/CD.

Q18 GitLab CI YAML deep dive - includes, extends, rules, needs, DAG

The Simple Answer

GitLab CI YAML has powerful features for code reuse, conditional execution, and pipeline optimization.

include - Importing Pipeline Definitions

include:local - From the same repository. **include:project** - From another GitLab project. **include:remote** - From any URL. **include:template** - From GitLab's built-in templates. This enables a central team to maintain shared pipeline templates that all projects import.

extends - Inheriting Job Definitions

Define a base job template with common configuration (Docker image, tags, artifacts). Other jobs extend it and override specific fields. This is DRY pipeline code - change the base once and all extending jobs inherit the change.

rules - Conditional Execution

Rules replaced the deprecated `only/except` keywords. They support complex conditions: rules with `if` for variable/branch conditions, rules with `changes` for file path matching, rules with `exists` for file existence. Rules are evaluated in order - the first matching rule determines the behavior.

needs - DAG Execution

By default, all jobs in a stage wait for the previous stage to complete. `needs` breaks this - a job can declare which specific upstream jobs it depends on, bypassing stage ordering. This dramatically increases parallelism. A deploy job can start as soon as build finishes, without waiting for unrelated linting in the same stage.

Q19 GitLab CI environments, deployment tracking, and Auto DevOps

The Simple Answer

GitLab CI has built-in environment tracking. When you define environment: name in a job, GitLab creates a deployment record. The Environments page shows what is deployed where, by whom, and when. You can rollback to any previous deployment directly from the UI.

Auto DevOps

Auto DevOps provides a zero-configuration CI/CD pipeline. Enable it and GitLab automatically detects your application language, builds a Docker image, runs tests, scans for security issues, and deploys to Kubernetes - all without writing a `.gitlab-ci.yml`. It uses Herokuish buildpacks for detection. Good for getting started quickly but most teams customize their pipeline eventually.

Review Apps

GitLab's review apps deploy every merge request to a dynamic environment. Combined with the Environments page, reviewers can click a link to see the change live. When the MR is merged or closed, the review app environment is automatically stopped and cleaned up.

Q20 GitLab CI merge trains - automated merge queues

The Simple Answer

Merge trains solve the problem of parallel merges breaking each other. Without merge trains, two MRs can both pass CI individually but fail when merged together because they modify conflicting code. Merge trains queue MRs and test each one merged with all previously queued MRs, ensuring the final result is always valid.

How Merge Trains Work

When MR-A joins the train, CI runs on `main + MR-A`. When MR-B joins, CI runs on `main + MR-A + MR-B` simultaneously. If MR-A passes, it merges. MR-B's result is already ready (since it was tested with MR-A included). If MR-A fails, MR-B is retested without MR-A. This parallel testing with sequenced merging prevents broken main branches while maintaining throughput.

Why this matters for interviews: Merge trains are GitLab's answer to GitHub's merge queue. Both solve the same problem - preventing 'merge skew' where individually passing PRs break each other when combined. Knowing this shows you understand the subtleties of CI at scale.

Q21 Docker in CI - building and pushing images

The Simple Answer

Building Docker images is the most common CI task. The pipeline checks out code, builds an image with a unique tag, pushes to a registry, and deploys.

Image Building Approaches

Docker-in-Docker (DinD) - Full Docker daemon inside a CI container. Requires privileged mode. Works everywhere but security concerns.

Docker socket mounting - Uses the host's Docker daemon. Faster but less isolated. See security implications in advanced questions.

Kaniko - Builds without a Docker daemon. Runs unprivileged. Designed for Kubernetes CI runners. Cannot run all Dockerfile instructions but covers most use cases.

BuildKit - Docker's modern build engine with better caching, secret handling, and parallel builds. Can run rootless.

Tagging Strategy

Always tag with the git SHA: myapp:abc123f. For releases, add semver: myapp:v1.2.3. Never use latest in CI - it is mutable and untraceable. The SHA makes every image traceable to an exact commit.

Q22 Basic testing in pipelines - unit, integration, smoke

The Simple Answer

Testing is the core value of CI. Without tests, CI is just an expensive way to compile code.

The Testing Pyramid

Unit tests (base - always, fast) - Test individual functions in isolation. Run in seconds. Cover 70-80% of cases. Run on every push.

Integration tests (middle - on PR) - Test component interaction - API endpoints, database queries. Run in minutes. Run on PRs and merges.

End-to-end tests (top - selective) - Full user flows. Slow, brittle, expensive. Run on merge or nightly.

Smoke tests (post-deploy) - Quick health checks after deployment - can the app start, connect to DB, respond to requests. Run after every deployment.

The rule: Fast tests gate every commit. Slow tests gate every PR. Full tests run nightly. Smoke tests validate every deployment.

Q23 Pull request pipelines and preview environments

The Simple Answer

PR pipelines are quality gates before merge. A good PR pipeline runs: linting (2 min), unit tests (5 min), integration tests (10 min), security scanning (5 min), build verification (5 min). Total: under 25 minutes. Enforce via branch protection - no merge without passing pipeline.

Preview Environments

Deploy every PR to an isolated URL like `pr-42.preview.example.com`. Reviewers test the change live. QA verifies before merge. Product managers see features before they ship. When the PR closes, the environment is destroyed.

Implementation: Kubernetes namespace per PR, Vercel/Netlify preview deploys, or Docker Compose on ephemeral VMs.

Preview environments transform code review. Instead of reading YAML and imagining how it looks, reviewers click a link and interact with the change. The cost of short-lived environments is far less than bugs discovered after merge.

Q24 Build agents/runners - self-hosted vs cloud-hosted

The Simple Answer

Cloud-hosted - Managed by the CI platform. Fresh VM per job. Zero maintenance. Limited customization. Good for most teams.

Self-hosted - On your infrastructure. More control over hardware, network, and software. More responsibility for security, patching, and scaling. Needed for GPU, internal network access, or compliance requirements.

Self-Hosted Security

A runner executes arbitrary code from your repository. If a malicious PR modifies the workflow, it runs on your runner. Never share runners between public and private repos. Use ephemeral runners (fresh container/VM per job). Rotate tokens regularly. Monitor for unexpected processes.

Q25 Notifications, dashboards, and pipeline reporting

The Simple Answer

Notify on failure (always), recovery (helpful), and production deployment (awareness). Do not notify on every success - it trains people to ignore notifications. Include commit message, author, branch, and a link to the failing step. Use dashboards (Grafana, DataDog) to track pipeline health trends across all services.

Q26 Concurrency control - preventing parallel deployments

The Simple Answer

Concurrency control ensures only one deployment runs at a time per environment. Without it, two simultaneous deployments can overwrite each other or both try to migrate the database at once.

Implementation

GitHub Actions - concurrency group with cancel-in-progress for builds, without cancel for deployments. **GitLab CI** - `resource_group` keyword. **Jenkins** - Lockable Resources plugin. All serialize deployments to prevent conflicts.

Q27 Rollback strategies in CI/CD pipelines

The Simple Answer

A rollback reverts production to a previous known-good state. The best strategy depends on your deployment method and whether database changes were made.

Approaches

Redeploy previous version - Deploy the last known-good image tag. Works when changes are backward-compatible and no schema changes were made.

Git revert and re-pipeline - Create a revert commit and push through the normal pipeline. Same testing guarantees. Slower but safer.

Kubernetes rollout undo - kubectl rollout undo. Instant. Works for K8s Deployments.

Blue-green switch - Flip traffic back to the old environment instantly.

Feature flag disable - Fastest rollback. No deployment needed. The code stays deployed but the feature is invisible.

Always know your rollback strategy before you deploy. Document it. Test it regularly. Include it in every deployment runbook. The worst time to figure out rollback is during a 3 AM incident.

Q28 Jenkins vs GitHub Actions vs GitLab CI - when to use which

The Simple Answer

Feature	Jenkins	GitHub Actions	GitLab CI
Best for	Complex enterprise pipelines	GitHub-native projects	GitLab-native projects
Pipeline language	Groovy	YAML	YAML
Hosting	Self-hosted only	Cloud or self-hosted	Cloud or self-hosted
Ecosystem	1800+ plugins	Marketplace actions	Built-in features
Learning curve	High	Low	Medium
Maintenance	High	Low	Medium
Built-in security scanning	No (plugin)	Limited (Dependabot)	Yes (SAST, DAST, dependency)
Container registry	No (plugin)	Yes (GHCR)	Yes (built-in)
Merge queues	No	Yes (merge queue)	Yes (merge trains)

The Decision Framework

Jenkins - Existing infrastructure, maximum customization, air-gapped environments, complex enterprise requirements. But prepare for high operational burden.

GitHub Actions - Code on GitHub, want zero-maintenance CI/CD, excellent developer experience, strong marketplace ecosystem.

GitLab CI - Use GitLab, want a single platform for code + CI/CD + security + deployment tracking. Best built-in security scanning.

Industry trend: New projects choose GitHub Actions or GitLab CI. Jenkins adoption is declining for new projects but existing installations persist due to migration cost.

SECTION 2 - Mid-Level Production Patterns (Q29-Q62)

Q29 Jenkins shared libraries - DRY pipeline code across 50+ teams

The Simple Answer

Shared libraries are reusable Groovy functions called from any Jenkinsfile. Instead of 50 teams copying the same Docker build logic, they call `buildDockerImage()` from a centrally maintained library. The library lives in a Git repo with `vars/` (global functions), `src/` (Groovy classes), and `resources/` (non-Groovy files).

How Pipelines Use Them

Pipelines import with `@Library('my-shared-lib@v1.2.0')` and call functions like `buildDockerImage(imageName: 'my-app')`. The library abstracts infrastructure complexity - teams provide parameters, the library handles implementation.

Governance

Shared libraries are infrastructure-critical. A bug breaks every pipeline. Version with Git tags. Pin versions in imports. Test changes in staging pipeline first. Breaking changes require major version bumps with migration guides.

Anti-pattern: Business logic in shared libraries. Libraries provide infrastructure primitives - build, test, deploy, notify. Application logic belongs in the Jenkinsfile.

Q30 Jenkinsfile advanced - post, when, parameters, input

post Blocks

always - Runs regardless of result. Cleanup. **success** - Notify on pass. **failure** - Alert on fail. **unstable** - Tests passed but quality gate failed. **changed** - Status changed from last build - useful for 'build is back to normal' notifications.

when Conditions

branch - `when { branch 'main' }` runs only on main. **changeset** - `when { changeset '/*.java' }` runs only when Java files changed. **expression** - `when { expression { return params.DEPLOY == true } }` runs based on custom logic. **environment**** - `when { environment name: 'ENV', value: 'prod' }` runs based on env vars.

parameters Block

Defines user-configurable inputs: string, choice, booleanParam, text. Parameters appear in the Jenkins UI when triggering manually. Essential for deployment pipelines where you choose the target environment or version.

input Step

Pauses the pipeline and waits for human approval. input message: 'Deploy to production?', submitter: 'deploy-team'. Blocks the pipeline until an authorized user approves. Used as a manual gate before production deployment.

Q31 GitHub Actions reusable workflows and composite actions

Reusable Workflows

Complete workflows in one repo that others call like functions. They accept inputs and secrets. They run as separate jobs with their own runner. Use for standardizing deployment pipelines across an organization.

Composite Actions

Step sequences bundled into a single action. Run within the calling job (same runner, same filesystem). Use for shared build steps like 'setup Node, install deps, lint.'

The Difference

Reusable workflows run as separate jobs - no shared filesystem. Composite actions run inline - share everything. Choose reusable workflows for complete pipelines. Choose composite actions for step-level reuse.

Practical Patterns

A central platform-workflows repo contains reusable workflows for deploy-to-kubernetes, publish-docker-image, and run-security-scan. Application repos call these with uses: org/platform-workflows/.github/workflows/deploy.yml@v1.0.0 passing inputs and secrets. The platform team maintains the workflows; application teams consume them.

Q32 GitHub Actions Docker actions vs JavaScript vs composite

Docker Actions

Run in a container defined by a Dockerfile. Can use any language. Isolated environment. Slower startup (must build or pull the container). Best for complex logic needing specific dependencies.

JavaScript Actions

Run directly on the runner in Node.js. Fastest startup. Access the GitHub toolkit libraries for API interactions. Best for GitHub-specific operations (commenting on PRs, creating releases).

Composite Actions

YAML-only sequences of steps. No additional language needed. Share the runner context with the calling workflow. Simplest to create and maintain. Best for grouping common step patterns.

Decision Guide

Need a specific runtime environment? Docker action. Need GitHub API interaction? JavaScript action. Need to bundle common steps? Composite action. When in doubt, start with composite - it is the simplest and covers most use cases.

Q33 GitHub Actions self-hosted runner groups, labels, and routing

The Simple Answer

Runner groups organize self-hosted runners by team or purpose. Labels tag runners with capabilities (gpu, linux, high-memory). Jobs request runners by label. This routes GPU jobs to GPU runners, sensitive builds to isolated runners, and general builds to shared runners.

Runner Groups

Groups restrict which repositories can use which runners. The 'production-deploy' group might only be available to the infrastructure repository. The 'general-build' group might be available to all repositories. This prevents untrusted repositories from accessing sensitive runners.

Actions Runner Controller (ARC)

The Kubernetes-based autoscaler for GitHub Actions runners. Runs runners as Pods. Scales up when jobs are queued, scales down to zero when idle. Each job gets a fresh Pod - perfect isolation. This is the industry standard for GitHub Actions self-hosted runners at scale.

Q34 GitLab CI parent-child pipelines and multi-project pipelines

Parent-Child Pipelines

A parent pipeline triggers child pipelines defined in separate YAML files. Each child runs independently with its own stages and jobs. Used in monorepos - the parent detects which directories changed and triggers only the relevant child pipelines.

Multi-Project Pipelines

A pipeline in one project triggers a pipeline in a different project. Used for deployment orchestration - the application project's CI triggers the deployment project's pipeline, which handles the actual rollout. This separates build concerns from deployment concerns.

Q35 Pipeline security - supply chain attacks and defenses

Attack Vectors

Dependency confusion - Malicious public package with the same name as your internal package. Your pipeline installs the public one. **Compromised action/plugin** - Third-party action hijacked. Every pipeline using it executes malicious code. **PR-based attacks** - Malicious PR modifies workflow to exfiltrate secrets. **Artifact tampering** - Build artifacts modified between build and deploy.

Defenses

Pin third-party actions by SHA. Use private package registry for internal packages. Use OIDC instead of stored credentials. Scan dependencies with Dependabot/Renovate/Snyk. Limit GITHUB_TOKEN permissions. Never run untrusted code with access to secrets. Sign artifacts with cosign. Verify provenance before deploying.

Q36 Secrets management - Vault, OIDC, short-lived credentials

OIDC - The Modern Approach

Instead of storing AWS access keys in your CI platform, configure AWS to trust the CI platform's OIDC identity. The pipeline requests a short-lived token that expires in minutes. No stored credentials. If the CI platform is compromised, there are no credentials to steal.

HashiCorp Vault Integration

For complex needs: dynamic secrets (unique database credentials per pipeline run), secret rotation, audit logging, and policy-based access. The pipeline authenticates to Vault using OIDC or AppRole, reads secrets, and uses them for the duration of the job.

The minimum standard: OIDC for cloud credentials, platform-managed secrets for everything else, secret scanning in CI to catch accidentally committed credentials. Long-lived access keys are technical debt.

Q37 Webhook security - signatures, verification, preventing spoofing

The Simple Answer

Webhooks trigger pipelines when external events occur (code push, PR created). Without verification, anyone who discovers your webhook URL can trigger pipelines by sending fake payloads.

How Webhook Signing Works

The webhook sender (GitHub, GitLab) includes a signature header computed using a shared secret (HMAC-SHA256). Your CI system verifies the signature before processing the payload. If the signature does not match, the webhook is rejected. This prevents unauthorized triggering.

Best Practices

Always configure a webhook secret. Verify signatures in your webhook handler. Use HTTPS for webhook URLs. Rotate webhook secrets periodically. Rate-limit webhook processing to prevent flood attacks. Log all webhook events for audit purposes.

Q38 Container image building in CI - Kaniko, BuildKit, buildx

Kaniko

Builds without a Docker daemon. Runs as an unprivileged container. Perfect for Kubernetes CI runners. Command: `/kaniko/executor --context=. --dockerfile=Dockerfile --destination=registry/image:tag`. Limitation: cannot run containers during build (no `RUN --mount=type=cache` for apt/pip).

BuildKit with Remote Cache

BuildKit supports cache export/import to registries. First build pushes cache. Subsequent builds pull cache and only rebuild changed layers. Combined with `--cache-to` and `--cache-from` flags, CI builds approach local build speeds.

Multi-Platform with buildx

`docker buildx build --platform linux/amd64,linux/arm64` produces images for both architectures from a single build. Essential when developers use Apple Silicon but production runs AMD64.

Q39 Infrastructure pipeline - Terraform plan/apply in CI/CD

The Pattern

PR opened → CI runs terraform plan → plan output posted as PR comment → human reviews plan → PR merged → CI runs terraform apply with saved plan file → results posted to Slack.

Critical Safety Measures

Never auto-apply to production. Always use `-out=planfile` and apply the saved file (what was reviewed is what gets applied). Gate production applies behind manual approval. Use separate state files and credentials per environment. Run `fmt` and `tflint` as pre-commit checks. Use Atlantis or Terraform Cloud for built-in approval workflows.

The number one Terraform CI disaster: Auto-applying without human review. A variable change that triggers database recreation goes unnoticed. Production data is destroyed. Always require manual approval for production Terraform applies. See the Terraform guide Q82 for the detailed scenario.

Q40 Database migrations in CI/CD - safe schema changes

The Expand-and-Contract Pattern

Expand - Add new column/table without removing old. Deploy code that writes to both. Old code still works.

Migrate - Backfill new column from old data. Runs as background job. **Contract** - Remove old column. Deploy code using only new schema.

Every deployment is backward-compatible. Rollback is always safe because old code works with current schema.

Never run destructive migrations in the same deployment as code changes. If the code rollback is needed, the migration has already destroyed data. Separate migration deploys from code deploys. Verify migrations are safe in staging before running in production.

Q41 End-to-end testing - Cypress, Playwright, Selenium

Playwright vs Cypress

Playwright - All browsers (Chromium, Firefox, WebKit). Faster execution. Better multi-page support. Auto-wait for elements. Becoming the default for new projects.

Cypress - Chromium-only (officially). Better developer experience with time-travel debugging. Stronger community. Declining relative to Playwright.

Best Practices

Run E2E against staging, not in PR pipeline (too slow). Limit to critical user journeys - login, purchase, core workflow. Parallelize across browsers. Implement retry for flaky tests. Track flake rates - fix or remove tests failing randomly above 1%.

Q42 Code quality gates - SonarQube, coverage thresholds, linting

The Simple Answer

Quality gates block merges when code does not meet standards. SonarQube analyzes code for bugs, vulnerabilities, code smells, and coverage. Coverage thresholds prevent untested code from shipping.

Practical Thresholds

Set minimum coverage on new code (not overall). Requiring 80% overall retroactively blocks teams with legacy codebases. Requiring 80% on new code gradually improves coverage without blocking legitimate work. Track code smells as warnings, not blockers - fix them in dedicated cleanup sprints.

Q43 SAST, DAST, and license scanning in CI/CD

SAST (Static Analysis)

Analyzes source code without running it. Fast. Catches SQL injection, XSS, hardcoded secrets, insecure deserialization. Tools: Semgrep, SonarQube, CodeQL, Snyk Code. Run on every PR.

DAST (Dynamic Analysis)

Tests the running application with malicious requests. Slower. Catches runtime vulnerabilities SAST misses. Tools: OWASP ZAP, Burp Suite. Run nightly against staging.

License Scanning

Detects open-source licenses in dependencies. Catches GPL-licensed dependencies that could create legal obligations for your proprietary code. Tools: FOSSA, Snyk, GitLab License Compliance. Run on every PR that changes dependency files.

Q44 Deployment strategies - blue-green, canary, rolling from CI/CD

Rolling

Replace instances gradually. Default in Kubernetes. Simplest. Risk: brief period with both versions serving traffic.

Blue-Green

Deploy new version alongside old. Switch traffic atomically. Instant rollback. Cost: doubles infrastructure during deployment.

Canary

Small percentage to new version. Monitor metrics. Gradually increase if healthy. Lowest risk. Most complex. CI pipeline's role: deploy artifact, configure traffic routing, monitor health, promote or rollback.

Q45 GitOps integration - CI builds, CD deploys via ArgoCD/Flux

The Workflow

Developer pushes code → CI builds image and pushes to registry → CI updates image tag in GitOps repo → ArgoCD detects change → ArgoCD deploys to cluster.

Why This Separation Matters

CI pipeline no longer needs cluster credentials. Only ArgoCD has deploy access. Git repository is the source of truth for deployments. Every deployment is a Git commit - auditable and reversible. If someone asks 'what is deployed in production,' the answer is always in Git.

Q46 ArgoCD Image Updater - automatic image tag updates

The Simple Answer

ArgoCD Image Updater watches container registries for new image tags and automatically updates the image reference in the GitOps repository. This eliminates the CI step that updates the GitOps repo - the Image Updater does it automatically when a new image appears.

How It Works

Configure an ArgoCD Application with image update annotations specifying the registry, image name, and tag strategy (semver, latest, digest). When a new matching image is pushed, Image Updater creates a commit in the GitOps repo updating the tag. ArgoCD then syncs the deployment.

Q47 Feature flags and CI/CD - decoupling deployment from release

The Simple Answer

Feature flags separate deployment (code on servers) from release (feature visible to users). Deploy incomplete features behind flags. Enable for internal testing. Gradually roll out. Instantly disable if problems arise. All without a deployment.

How It Changes CI/CD

With feature flags, trunk-based development is safe. Incomplete features go to main behind flags. CI deploys continuously. Product controls feature visibility through a dashboard (LaunchDarkly, Flagsmith, Unleash). Rollback becomes 'disable the flag' - no redeployment needed.

Q48 Pipeline caching strategies - reducing build times by 80%

Three Layers of Caching

Dependency cache - npm, pip, Maven packages. Key: hash of lock file. Re-download only when dependencies change.

Build cache - Docker layers, compiled artifacts. Use BuildKit --cache-from registry. Only changed layers rebuild.

Test cache - Skip unchanged test suites. Nx, Turborepo, Bazel analyze the dependency graph and run only affected tests.

Results

Dependency caching: 5 minutes saved per build. Docker layer caching: 5-10 minutes saved. Test caching in monorepo: 10+ minutes saved. Combined: 20+ minutes saved, bringing 25-minute pipelines to under 5 minutes.

Q49 Parallelism and matrix builds

Matrix Builds

Test across Node 18, 20, 22 on Ubuntu and macOS. Six combinations from one job definition, running simultaneously. Completes in the time of the slowest single combination, not the sum of all six.

Test Parallelism

Split a test suite across multiple runners. Jest --shard, pytest-split. 1000 tests sequentially in 15 minutes become 1000 tests across 4 shards in 4 minutes.

Q50 Approval gates and deployment windows

Approval Gates

GitHub: environments with required reviewers. GitLab: when: manual. Jenkins: input step. Pipeline pauses, sends notification, waits for human approval.

Deployment Windows

Prevent deployments during peak hours, holidays, or weekends. Implement with: pipeline conditionals checking the current time, a deployment freeze ConfigMap in Kubernetes, or a feature in your deployment platform (ArgoCD sync windows). Emergency overrides must exist but require additional approval and create an audit trail.

Q51 Pipeline templates and standardization

The Simple Answer

Templates provide a standardized pipeline that includes all quality gates, security scanning, and deployment best practices. Teams customize for their needs without reinventing infrastructure.

Implementation

GitHub - Reusable workflows in a central org/platform-workflows repo. Application repos call them with uses:.

GitLab - include templates from a shared project. Templates define standard stages. Projects extend with overrides.

Jenkins - Shared libraries provide standardized functions. A template Jenkinsfile calls library functions in the standard order.

The goal is not forcing everyone into the same pipeline. It is making the right thing the easy thing. A new service gets a production-ready pipeline in 5 minutes by using the template. Deviations are allowed but documented.

Q52 Test result reporting - JUnit XML, flaky test detection

JUnit XML Format

Most CI platforms parse JUnit XML for test result display. Test frameworks output results in this format (Jest --reporters=jest-junit, pytest --junitxml=results.xml). The CI shows pass/fail counts, duration, and failing test details directly in the UI.

Flaky Test Detection

Track pass/fail per test across runs. A test passing 95% of the time is flaky. Quarantine tests exceeding 1% flake rate - move them to a non-blocking pipeline. Fix within a sprint. Delete at 5% if not fixed. Tools: BuildPulse, Trunk.io, or custom scripts analyzing JUnit XML history.

A flaky test blocking the pipeline erodes team trust in CI. Developers start re-running pipelines 'just in case,' wasting time and money. Eventually they bypass CI entirely. Treat flaky tests as production bugs with the same urgency.

Q53 Ephemeral environments - preview environments per PR

Implementation Approaches

Kubernetes namespace per PR - Create namespace, deploy PR image, expose via Ingress with a PR-specific hostname. Destroy on PR close. Tools: ArgoCD PR Generator, custom scripts.

Vercel / Netlify - Preview deployments natively for web frontends. Zero configuration.

Docker Compose on ephemeral VMs - Spin up a VM, run the PR stack, assign DNS. Destroy on PR close.

Q54 Versioning and tagging - semver, calver, semantic-release

Strategies

Semantic Versioning - Major.Minor.Patch. Breaking.Feature.Fix. Best for libraries and formal releases.

CalVer - Calendar-based (2025.06.15). Best for continuously deployed applications.

Git SHA - Universal internal identifier. Every image traceable to a commit.

Automated Versioning with semantic-release

Analyzes commit messages (Conventional Commits: feat:, fix:, BREAKING CHANGE:). Automatically determines next version, creates Git tag, updates changelog, and publishes release. Eliminates manual versioning errors. Setup: install semantic-release, configure in CI to run on main branch merges, use Conventional Commits in your team.

Q55 GitHub Actions security - permissions, GITHUB_TOKEN, OIDC

GITHUB_TOKEN Permissions

Set permissions: read-all at workflow level. Grant specific writes where needed. Default token has more access than most workflows need. Restrict to minimum: contents: read, packages: write (for publishing), id-token: write (for OIDC).

Pin Actions by SHA

uses: actions/checkout@abc123 not @v4. Tags are mutable - a compromised tag injects malicious code. SHAs are immutable. Use Dependabot to auto-update pinned SHAs.

OIDC for Cloud

Configure AWS/GCP/Azure to trust GitHub OIDC. Get short-lived credentials scoped to specific repos and branches. No stored secrets. A compromised repo can only request tokens for itself, not other repos.

Fork Attack Prevention

Never use pull_request_target to run untrusted code with secrets. Use pull_request for CI on external PRs. If you need PR comments from fork PRs, use a two-job pattern: Job 1 (no secrets) builds and tests. Job 2 (triggered by workflow_run) runs trusted code with secrets.

Q56 Monorepo CI - detecting changes, building only what changed

Path-Based Triggers

GitHub Actions: on.push.paths. GitLab CI: rules with changes. Jenkins: changeset detection in when blocks. A commit to service-a/ triggers only service-a's pipeline.

Build-System Level Detection

Nx, Turborepo, Bazel analyze the dependency graph. A change to a shared library triggers builds for all dependent services but not unrelated ones. Much smarter than path-based triggers because it understands cross-package dependencies.

Without change detection, monorepo CI build times grow linearly with service count. 50 services with 10-minute pipelines = 500 minutes of compute per commit. With detection: only 10 minutes for the affected service.

Q57 Pipeline timeout, retry strategies, and idempotency

Timeouts

Set at job and step level. A 10-minute timeout on a build step catches infinite loops. A 30-minute timeout on the entire job prevents runaway pipelines. Always set timeouts - without them, a hung build consumes a runner indefinitely.

Retry Strategies

Retry on transient failures (network timeouts, flaky infrastructure). Do not retry on deterministic failures (syntax errors, test failures). Implement exponential backoff for external API calls. Limit retries to 2-3 attempts - more suggests a real problem.

Idempotency

Running the same pipeline twice should produce the same result. Use immutable tags (SHA-based). Use atomic deployments (replace, do not patch). Avoid stateful operations in pipelines that break on re-run (like creating a resource that already exists - use create-or-update patterns).

Q58 Build reproducibility - hermetic builds, deterministic outputs

The Simple Answer

A reproducible build produces identical output from identical input regardless of when or where it runs. The same commit should always produce the same artifact.

Threats to Reproducibility

Unpinned dependency versions (npm install without a lockfile). Build tools that include timestamps. Environment-specific variables leaking into artifacts. Floating Docker base image tags (FROM python:3 resolves to different images over time).

Achieving Reproducibility

Pin all dependencies with lock files. Pin Docker base images by digest (@sha256:abc123). Pin build tool versions. Avoid environment-dependent build steps. Use hermetic builds (Bazel) that explicitly declare all inputs and cannot access network or host filesystem.

Q59 Container registry integration - ECR, GCR, Harbor

ECR (AWS)

Authenticate with `aws ecr get-login-password` or OIDC. Tokens expire every 12 hours - use credential helpers or CronJobs for rotation. Configure lifecycle policies to clean up old images. Enable scanning-on-push for vulnerability detection.

GCR / Artifact Registry (Google)

Authenticate with service account key or Workload Identity Federation (OIDC). Artifact Registry supports multi-format (Docker, npm, pip, Maven). Vulnerability scanning built-in.

Harbor (Self-Hosted)

Open-source registry by CNCF. Provides scanning (Trivy), signing (Notary/cosign), replication, RBAC, and audit logging. Good for organizations needing full control or on-premises requirements. Can act as a pull-through cache for Docker Hub, reducing rate limit impact.

Q60 Helm and Kubernetes deployments from CI/CD

Direct Deployment

Pipeline runs `helm upgrade --install --set image.tag=$SHA`. Simple. Pipeline needs cluster credentials. No Git audit trail.

GitOps Pattern (Recommended)

Pipeline updates image tag in GitOps repo. ArgoCD/Flux deploys. CI never touches the cluster. Better security (CI has no cluster access) and auditability (Git history = deployment history).

Helm Best Practices in CI

Always use `--timeout` and `--wait`. Use `--atomic` for automatic rollback on failure. Pin Helm chart versions. Test with helm template before deploying. Use helm diff plugin to preview changes.

Q61 Multi-environment promotion - dev to staging to prod

The Principle

Build once, deploy to many environments. The same image (same SHA tag) goes through dev → staging → prod. Environment differences are handled through configuration (ConfigMaps, env vars), not different builds.

Pipeline Flow

Merge to main → auto-deploy to dev → automated tests pass → auto-deploy to staging → integration tests pass → manual approval → deploy to production → smoke tests pass → done.

Anti-pattern: Different build processes per environment. If staging builds differently from production, staging tests prove nothing about production behavior.

Q62 Pipeline cost optimization - reducing CI/CD spend

Where Money Goes

Compute (runner minutes) is the biggest cost. Storage (artifacts, caches, images) is secondary. Tool licenses (SonarQube, security scanners) are fixed costs.

Optimization Playbook

Cache aggressively - Dependencies, Docker layers. 50-80% build time savings. **Right tests at right time** - Fast on push, full on PR, E2E nightly. **Spot instances** - 60-80% cheaper for self-hosted runners. **Right-size runners** - Small

for lint, large for Docker builds. **Skip unchanged** - Monorepo change detection. **Fix triggers** - Do not run full pipelines on every push to every branch. **Clean up** - Delete old artifacts, prune caches, set TTLs.

SECTION 3 - Senior Architecture & Advanced (Q63-Q84)

Q63 CI/CD architecture at scale - 500+ developers, 100+ services

The Simple Answer

At scale, CI/CD is a platform engineering challenge. Pipelines must be fast (under 10 minutes), reliable (99.9% success rate for infrastructure failures), and cost-efficient. This requires centralized templates, autoscaling build infrastructure, multi-level caching, and observability.

Architecture Decisions

Centralized templates + team customization - Platform team maintains golden path templates. Application teams customize for their needs. Deviations are allowed but tracked.

Build infrastructure - Autoscaling runner pools by workload type (small for lint, large for Docker, GPU for ML). Ephemeral runners for isolation. Cache servers close to runners.

Artifact management - Single registry with lifecycle policies. Promotion-based workflow - same artifact promoted through environments, never rebuilt.

Self-service - Teams create pipelines without platform team involvement. Templates make the right thing the easy thing. Guardrails prevent dangerous patterns without blocking progress.

Q64 Jenkins at scale - HA, shared library governance, plugin management

High Availability

Jenkins controller HA uses active-passive with shared storage or controller federation (multiple controllers with shared agent pools). The CloudBees CI product provides HA and controller federation natively. Open-source Jenkins HA requires custom setup with shared NFS/EFS storage and a load balancer.

Plugin Management

Curate a tested plugin list. Test plugin upgrades in a staging Jenkins before production. Use the Jenkins Plugin Manager tool to define plugins as code. Monitor Jenkins security advisories - plugin vulnerabilities are the most common Jenkins security issue.

Shared Library Governance

Version with semver. Test in a dedicated pipeline before release. Deprecation periods for breaking changes (minimum 30 days). Migration guides for major versions. Dedicated Slack channel for library announcements.

Honest assessment: Jenkins at scale requires 0.5-2 FTE for operations. Plugin management, controller upgrades, agent scaling, credential rotation, and security patching are ongoing responsibilities. This is the primary driver of migration to managed platforms.

Q65 Pipeline observability - metrics, traces, DORA metrics

DORA Metrics

Deployment Frequency - How often you deploy. Elite: multiple times per day. **Lead Time** - Commit to production. Elite: under one hour. **Change Failure Rate** - Deployments causing failures. Elite: under 5%. **MTTR** - Time to restore service. Elite: under one hour.

Pipeline-Specific Metrics

Pipeline success rate (target 99% for non-code failures). Duration trends (alert on regressions). Queue time (how long jobs wait for runners). Flaky test rate. Cache hit rate. Cost per pipeline run.

Implementing Observability

Export pipeline metrics to Prometheus/Grafana. Track trends over weeks and months. Set alerts on regressions. Use the data to prioritize improvements - if queue time is 5 minutes, you need more runners. If cache hit rate is 40%, fix your cache keys.

Q66 Compliance and audit - SOC2, HIPAA, change management

How CI/CD Enables Compliance

Traceability - Every deployment links to a commit → PR → ticket. The entire chain is auditable. **Separation of duties** - Developers write, reviewers approve, pipeline deploys. No single person makes unreviewed production changes. **Immutable artifacts** - Build once, deploy everywhere. Image signing proves the artifact is untampered. **Audit logs** - Every pipeline run, approval, and deployment is logged with timestamps and actors.

Practical Implementation

Require PR reviews for all production changes. Require pipeline success for all merges. Log all deployments with actor, timestamp, artifact version, and approval chain. Retain logs for the compliance-required period. Sign all artifacts. Use OIDC to eliminate stored credentials. Implement deployment freeze windows for change control.

Q67 Pipeline security hardening - SLSA levels, Sigstore

SLSA Framework

Level 1 - Documented build process. Level 2 - Hosted build service (not developer laptop). Level 3 - Hardened build platform with tamper-proof provenance. Level 4 - Two-party review and hermetic builds.

Sigstore

cosign - Signs container images with keyless signing tied to OIDC identity. **Rekor** - Transparency log recording all signatures. **Fulcio** - Certificate authority issuing short-lived signing certificates.

Together they prove an artifact was built by a specific CI pipeline from a specific commit. No long-lived signing keys to manage or protect.

Q68 Multi-cloud CI/CD

The Pattern

Build stage is cloud-agnostic - produces a Docker image. Test stage is cloud-agnostic. Deploy stage branches per cloud - each branch authenticates via OIDC and deploys using cloud-specific tooling (EKS, GKE, AKS). The artifact is identical across clouds; only the deployment mechanism differs.

Q69 Migrating from Jenkins to GitHub Actions or GitLab CI

Strategy

Phase 1 - Inventory all Jenkins jobs. Classify by complexity. Map plugins to platform features. **Phase 2 - Create templates** for common patterns (build-test, build-test-deploy, infrastructure). **Phase 3 - Migrate simple jobs first** (60% of jobs). Build confidence and patterns. **Phase 4 - Migrate complex jobs** including shared library logic. **Phase 5 - Parallel run** both systems. Compare results. Decommission Jenkins when confident.

Timeline

2 engineers can migrate 200 jobs in 3-4 months. Month 1: templates and pilots. Months 2-3: bulk migration. Month 4: complex long tail and decommissioning.

Pitfalls

Shared library migration is the hardest part. Build history does not transfer. Runner environments must be recreated. Secrets must be regenerated, not copied.

Q70 Advanced rollback - database-aware, feature-flag, traffic-based

Database-Aware Rollback

If a deployment included a migration, the old code must work with the new schema (expand-and-contract). Rollback the code but keep the new schema. The old code was designed to work with both old and new schema.

Feature-Flag Rollback

Disable the flag instead of redeploying. Instant. No infrastructure changes. The code stays deployed but the feature is invisible. Fastest rollback possible.

Traffic-Based Rollback

In canary deployments, route 100% back to the old version. New version Pods still run but receive no traffic. Diagnose, fix, re-canary.

Q71 CI/CD for microservices - contracts, dependency ordering

Contract Testing

Consumer-Driven Contracts (Pact) define expected API contracts between services. The consumer defines expectations. The provider verifies. Contracts checked in CI - a provider change breaking a contract fails the pipeline before deployment.

Deployment Ordering

Services with no dependencies deploy first. Dependent services deploy after their dependencies are verified. Managed through pipeline dependencies or a deployment orchestrator.

Q72 Trunk-based development at scale

Requirements

Branches live less than 24 hours. CI pipeline runs under 10 minutes. Feature flags for incomplete work. Culture where fixing the build is top priority. You need a fast, reliable pipeline first - if CI takes 30 minutes, developers will not merge frequently.

Q73 Tekton Pipelines - Kubernetes-native CI/CD

The Simple Answer

Tekton defines CI/CD pipelines as Kubernetes Custom Resources. Tasks, Pipelines, PipelineRuns, and TaskRuns are all Kubernetes objects. Each step runs in a container. Pipelines execute as Kubernetes Pods. This makes CI/CD a native workload on your cluster rather than a separate system.

When to Use Tekton

Teams already running Kubernetes that want CI/CD as a platform capability. Organizations building internal developer platforms where CI/CD needs to be deeply integrated with Kubernetes. Companies that want to avoid external CI/CD dependencies.

When NOT to Use Tekton

Teams that want a managed CI/CD service (GitHub Actions, GitLab CI are easier). Small teams without Kubernetes expertise. Projects where the CI/CD pipeline is simple and does not justify the Tekton learning curve.

Q74 Pipeline as a product - internal developer platforms

The Simple Answer

An internal developer platform abstracts CI/CD complexity behind self-service interfaces. Developers do not write YAML - they register a service and get a production-ready pipeline. Tools: Backstage (Spotify), Port, Humanitec. The platform team builds and maintains the golden path; developers consume it.

Q75 Disaster recovery for CI/CD infrastructure

CI/CD is Critical Infrastructure

If CI/CD goes down, deployments stop, hotfixes cannot ship, incidents cannot be resolved. DR planning: pipeline definitions in Git (automatic). CI platform configuration backed up. Credentials documented securely. Emergency deployment procedure documented (manual kubectl/helm as break-glass). Secondary CI platform or manual procedure for critical hotfixes.

Q76 Progressive delivery - Argo Rollouts, Flagger

The Simple Answer

CI triggers a progressive delivery tool instead of direct deployment. Argo Rollouts or Flagger manage canary progression - automatically advancing traffic, checking metrics, rolling back if error rates exceed thresholds. CI initiates; the tool manages the multi-step promotion.

Q77 Release management - semver, changelogs, release trains

Release Trains

Ship on a fixed schedule regardless of what is ready. Provides predictability for downstream teams and customers. Missed features go to the next train. Reduces coordination overhead - everyone knows the next release date.

Automated Changelogs

semantic-release generates changelogs from Conventional Commits. Each release automatically documents features, fixes, and breaking changes. No manual changelog writing.

Q78 CI/CD for mobile apps

Unique Challenges

Platform-specific builds (Xcode for iOS, Android SDK). Code signing management. App store submission and review. Cannot force-update user devices. Mobile pipelines use specialized runners (macOS for iOS), manage signing certificates securely, automate submissions through Fastlane, and use feature flags for soft rollbacks.

Q79 Zero-downtime deployment verification

Post-Deployment Verification Chain

Health check endpoints (immediate). Smoke tests hitting critical endpoints (30 seconds). Synthetic monitoring running full user flows (5 minutes). Error rate monitoring comparing to pre-deployment baseline (15 minutes). If any verification fails, trigger automatic rollback.

Q80 Cross-tool pipeline triggers - webhook chaining

The Simple Answer

Complex organizations use multiple CI/CD tools - Jenkins for legacy, GitHub Actions for new projects, GitLab CI for infrastructure. Webhook chaining connects them: GitHub push → GitHub Actions builds image → webhook triggers Jenkins deployment pipeline → Jenkins triggers GitLab CI for infrastructure validation. Use API calls or webhook relays. Document the chain. Monitor end-to-end latency.

Q81 CircleCI, Azure DevOps, Bitbucket Pipelines

CircleCI

Strong Docker support. Orbs (reusable packages) similar to GitHub Actions marketplace. Good caching and parallelism. Popular with startups.

Azure DevOps

Microsoft's CI/CD platform. Tight Azure integration. Multi-stage YAML pipelines. Good for organizations invested in Microsoft ecosystem. Supports both cloud-hosted and self-hosted agents.

Bitbucket Pipelines

Atlassian's CI/CD. Integrated with Jira and Confluence. Simple YAML syntax. Good for teams using Bitbucket for source code.

Q82 AI in CI/CD

Predictive Test Selection

ML models predict which tests are likely affected by a change. Run only those tests. Reduces test time by 80%. Tools: Launchable, Trunk.io.

Flaky Test Detection

Pattern analysis across test runs identifies non-deterministic tests. Automatic quarantine prevents pipeline blocking.

Auto-Remediation

Automatically retry known transient failures (network timeouts, rate limits). Auto-create issues for persistent failures. Future: AI-generated fix suggestions for common failure patterns.

Q83 Change freeze and deployment windows

The Simple Answer

Change freezes prevent deployments during high-risk periods - Black Friday for e-commerce, tax season for finance, holidays. Deployment windows restrict when deployments can happen - business hours only, not Fridays after 3 PM.

Implementation

Pipeline conditionals checking current time and a freeze schedule. ArgoCD sync windows (prevent syncing during blackout periods). Kubernetes admission webhooks rejecting deployments during freezes. Emergency override requires additional approval and creates audit trail.

Q84 Pipeline as DAG vs linear stages

The Simple Answer

Traditional pipelines run stages linearly: build → test → deploy. DAG (Directed Acyclic Graph) pipelines express dependencies between individual jobs, not stages. Jobs run as soon as their dependencies complete, maximizing parallelism.

DAG Example

Build-backend and build-frontend run in parallel (no dependencies). Test-backend depends on build-backend. Test-frontend depends on build-frontend. Deploy depends on both tests. Total time = max(build+test paths) instead of sum of all stages.

Tool Support

GitLab CI: needs keyword. GitHub Actions: needs keyword. Jenkins: parallel stages. Tekton: native DAG execution. Modern pipelines should always use DAG when possible.

SECTION 4 - Tricky Scenarios & Real-World Disasters (Q85-Q110)

Q85 Pipeline takes 45 minutes - optimize to under 10

Step-by-Step Diagnosis

Step 1 - Profile each stage. Break down: checkout (seconds), dependency install (should be cached), build (should be incremental), tests (usual bottleneck), Docker build (should use cache), deploy (should be fast).

Step 2 - Fix dependency installation. If npm install takes 5 minutes every run, caching is broken. Verify cache key includes lock file hash. Verify cache restore in logs. Working cache: 5 minutes → 10 seconds.

Step 3 - Fix test execution. If tests take 20 minutes, parallelize across 4 runners. Use test impact analysis to skip unrelated tests. Quarantine flaky tests causing retries.

Step 4 - Fix Docker builds. Check instruction ordering - dependencies before source code. Use BuildKit with --cache-from registry. Multi-stage parallelization.

Step 5 - Run stages in parallel. Lint, unit tests, and security scans are independent - run simultaneously as a DAG.

Step 6 - Skip unchanged. In monorepos, detect which packages changed. Only build and test affected packages.

Typical Results

Caching: -5 min. Test parallelism: -10 min. Docker cache: -5 min. DAG parallelism: -5 min. Total: 25 minutes saved, bringing 45-minute pipelines to under 20. Test selection + monorepo detection can reach under 10.

Q86 Production deployment broke - rollback with database changes

What Happened

Deployment included code + database migration renaming a column (old_name → new_name). New code references new_name. After deployment, application errors. Need rollback but old code needs old_name which no longer exists.

Step-by-Step Resolution

Step 1 - Assess damage. Is the app completely broken or partially degraded? How many users affected?

Step 2 - If immediate rollback needed. Run reverse migration adding old column back: ALTER TABLE t ADD COLUMN old_name TEXT; UPDATE t SET old_name = new_name; Then rollback application code. Both columns now exist.

Step 3 - Fix root cause. Should have used expand-and-contract: first deploy adds new_name and writes to both. Second deploy migrates reads. Third deploy removes old_name.

Prevention

Never rename/delete columns in the same deployment as dependent code changes. Every migration must be backward-compatible with currently deployed code. Test rollbacks in staging.

Q87 Secret leaked in CI logs - incident response

Step-by-Step

Step 1 - Rotate immediately. Change password, regenerate API key. Assume compromised the moment it appeared.

Step 2 - Identify exposure scope. Who can see CI logs? Are logs in centralized logging (Elasticsearch, Datadog)? Check artifact storage.

Step 3 - Purge logs. Delete workflow run logs. Purge from centralized logging. Delete artifacts containing secrets.

Step 4 - Find how it leaked. Debug step printing env vars? App logging config at startup? Test framework logging auth headers? Script echoing variables?

Step 5 - Prevent. Enable secret masking. Add log scanning for secret patterns. Never print environment variables. Use OIDC so even leaked tokens expire in minutes.

Most CI secret leaks are from echo \$DATABASE_URL for debugging. Education and automated scanning are the primary defenses.

Q88 CI runner compromised - supply chain attack

Step-by-Step

Step 1 - Isolate runner. Remove from pool. Disconnect network. Preserve for forensics.

Step 2 - Assess exposure. What secrets did the runner have? What artifacts did it produce? What deployments did it trigger? Rotate everything.

Step 3 - Audit recent builds. Check all jobs from past 7-30 days. Verify artifact checksums. Check for tampered deployments.

Step 4 - Verify deployed artifacts. If using cosign, verify signatures. If not signed, rebuild and redeploy from trusted runner.

Step 5 - Rebuild runner. Fresh from clean image. Never clean a compromised runner - treat as permanently untrusted.

Prevention

Ephemeral runners (fresh per job). SLSA Level 3 practices. Sign all artifacts. Monitor runners. OIDC instead of stored credentials.

Q89 Flaky test blocking all deployments for a week

Step-by-Step

Step 1 - Identify by reviewing pipeline history. Sort by failure rate.

Step 2 - Quarantine immediately. Move to non-blocking pipeline. Main pipeline passes. Deployments unblocked.

Step 3 - Diagnose root cause. Timing dependency? Test ordering? External service? Shared mutable state?

Step 4 - Fix or delete. Make deterministic (replace sleeps with deterministic waits, mock externals, isolate state). If unfixable after one sprint, delete it.

Step 5 - Implement flake detection. Track per-test results. Auto-quarantine at 1%. Alert at 2%. Delete at 5%.

Common causes by frequency: Timing/race conditions (40%), external service flakiness (25%), shared state between tests (20%), test ordering dependency (15%).

Q90 Two teams deployed conflicting changes simultaneously

Resolution

Step 1 - Identify which caused the issue. Check deployment logs and timestamps.

Step 2 - Rollback the breaking change (usually the API change that broke consumers).

Step 3 - Implement API versioning (v1, v2) so breaking changes coexist with old versions.

Prevention

Deployment concurrency control - one deploy per service at a time. Contract testing between dependent services. API versioning. Deployment coordination channel.

Q91 Jenkins controller disk full - builds failing

Step-by-Step

Step 1 - Find consumption. Check `/var/jenkins_home`. Culprits: build workspaces, archived artifacts, unrotated logs, Docker cache.

Step 2 - Cleanup. Discard Old Builds on all jobs. Remove artifacts older than 30 days. Compress logs. Clean Docker cache.

Step 3 - Prevent. Default Discard Old Builds (keep 10). External artifact storage (S3, Artifactory). Log rotation. Docker builds on agents not controller. Disk monitoring with alerting at 80%.

Q92 GitHub Actions fork stealing secrets via `pull_request_target`

What Happened

Malicious fork modified workflow using `pull_request_target` (runs with base repo secrets) to exfiltrate credentials.

Response

Step 1 - Revoke all secrets. Rotate every credential. **Step 2 - Audit logs** for suspicious API calls. **Step 3 - Fix workflow.** Replace `pull_request_target` with `pull_request` for untrusted code. If needed, use two-job pattern: Job 1 (no secrets) builds fork code. Job 2 (`workflow_run`, with secrets) runs only trusted base branch code.

Prevention

Never run fork code with secret access. Use `pull_request` for CI. Restrict `GITHUB_TOKEN`. Review all uses of `pull_request_target` in your org.

Q93 Deployment succeeded but app is broken - no rollback plan

Step-by-Step

Step 1 - Deploy previous image. Find last good tag from deployment history. `kubectl set image` or `helm upgrade` with previous tag. Emergency - skip pipeline.

Step 2 - If image unavailable. Find commit SHA. Trigger build from that commit. While waiting, revert the Git commit.

Step 3 - Implement rollback for next time. Document procedure in runbook. Automate to one-click/command. Test in staging quarterly.

Prevention

Every deployment needs: documented rollback procedure, one-click mechanism, smoke tests verifying business logic (not just health endpoints), synthetic monitoring detecting logic bugs.

Q94 Pipeline costs tripled - audit and optimize

Step-by-Step

Step 1 - Check billing. Break down by repo, workflow, runner type. Look for new repos, increased frequency, longer durations.

Step 2 - Find runaways. Misconfigured trigger (every push instead of PRs)? Pipeline in retry loop?

Step 3 - Check caching. Invalid cache key causing full rebuilds every run?

Step 4 - Optimize. Restore cache. Fix triggers. Right-size runners. Spot instances. Skip unchanged packages.

Q95 Helm upgrade failed midway - release stuck

Step-by-Step

Step 1 - helm history shows pending-upgrade revision. **Step 2 - helm rollback** to last deployed revision. **Step 3 - If rollback fails**, delete the pending-upgrade Secret. **Step 4 - Re-run upgrade.**

Prevention

Always `--timeout 10m --wait --atomic` in CI Helm commands. `--atomic` auto-rolls back on failure. Set pipeline timeout longer than Helm timeout.

Q96 Feature branch merged without passing CI - broken main

Step-by-Step

Step 1 - Revert immediately. `git revert -m 1 MERGE_SHA && git push`. Main is green again.

Step 2 - Developer fixes on their branch. Reverts the revert when ready.

Step 3 - Fix branch protection. Required status checks. Required reviews. No bypass for admins.

Branch protection is non-negotiable. A broken main blocks the entire team. The cost of skipping CI is always higher than waiting.

Q97 Docker image wrong architecture - crashes in prod

What Happened

Image built as linux/arm64 (Apple Silicon runner). Production runs linux/amd64. Container crashes with exec format error.

Fix

Specify `--platform linux/amd64` in CI builds. Or use multi-platform buildx for both architectures. Add CI step verifying image architecture before push. Never rely on runner's native architecture matching production.

Q98 Terraform apply in CI destroyed production

Step-by-Step

Step 1 - Restore from backup. Check RDS snapshots, Cloud SQL backups.

Step 2 - Assess blast radius. What else was affected?

Step 3 - Fix pipeline. Never auto-apply to production. Require manual approval between plan and apply. Always use saved plan files. Add `prevent_destroy` on critical resources.

Terraform production applies MUST require human approval. This is the most common infrastructure CI/CD disaster and the most preventable one.

Q99 CI pipeline leaking credentials through artifacts

What Happened

Build step generates debug report including all environment variables. Report uploaded as artifact. Anyone with download access reads AWS credentials.

Response

Rotate credentials immediately. Delete artifacts. Use OIDC so leaked tokens are expired. Never include env dumps in artifacts. Sanitize reports before upload.

Q100 Coordinated deployment to 50 microservices

Step-by-Step

Step 1 - Categorize by dependency. Independent services first. Dependent services after their dependencies. Create a dependency graph.

Step 2 - Deploy in waves. Wave 1: 5 lowest-risk, no dependencies. Verify. Wave 2: 15 moderate. Wave 3: 20 remaining. Wave 4: 10 critical with most dependencies. Each wave includes verification and rollback plan.

Step 3 - Feature-flag the change. If possible, deploy updated library to all services with flag disabled. Enable flag gradually per service. Rollback = disable flag.

Step 4 - Automate with Argo Rollouts or a custom orchestrator that tracks the dependency graph, deploys in order, verifies health, and halts on failure.

Prevention

Avoid breaking changes in shared libraries. Semver - non-breaking (minor/patch) deploy independently. Breaking (major) requires this coordinated approach. Backward-compatible changes eliminate coordination.

Q101 Migrating 200 Jenkins jobs to GitHub Actions

Step-by-Step

Step 1 - Inventory all 200 jobs. Simple (build+test, 60%), medium (build+test+deploy, 30%), complex (shared libraries, custom plugins, 10%).

Step 2 - Create standard workflow templates. build-and-test for each language, deploy-to-k8s, publish-docker. Most simple jobs are instances of these.

Step 3 - Migrate in waves. 20 per week. Run Jenkins and Actions in parallel. Compare results. Disable Jenkins job when confident.

Step 4 - Migrate shared library logic. Library functions become reusable workflows or composite actions. Complex Groovy often simplifies significantly.

Step 5 - Handle the long tail. Last 10% take 50% of effort. Some need custom actions replacing plugin functionality.

Pitfalls

Underestimating shared library migration. Losing build history. Runner environment differences. Secret regeneration (never copy secrets between systems).

Timeline

2 engineers, 3-4 months. Month 1: templates and pilots. Months 2-3: bulk migration. Month 4: complex tail and decommissioning.

Q102 Jenkins shared library update broke 50 team pipelines

What Happened

The platform team pushed a breaking change to the shared library's main branch. Fifty teams had `@Library('shared-lib')` _ without version pinning. Their next pipeline run pulled the breaking change and failed.

Step-by-Step Resolution

Step 1 - Revert the library change immediately. The library's main branch returns to the previous working state. Teams' next pipeline runs succeed.

Step 2 - Communicate the incident. Notify all affected teams via Slack/email that the issue is resolved and what happened.

Step 3 - Re-introduce the change properly. Create a new major version (v3.0.0) with the breaking change. Document migration steps. Update teams gradually with deprecation warnings on the old version.

Step 4 - Fix the governance. All pipeline imports must pin versions: `@Library('shared-lib@v2.5.0')`. The library's main branch should be a development branch, not what production pipelines pull. Release through tags only.

Prevention

Version-pin all library imports. Never import from `@main`. Test library changes in a staging pipeline first. Breaking changes require major version bumps with migration guides and 30-day deprecation periods. Treat shared libraries like public APIs.

Q103 workflow_dispatch with wrong inputs deployed wrong branch to prod

What Happened

An engineer manually triggered a GitHub Actions workflow using `workflow_dispatch`. The workflow had a branch input with a default of `main`. The engineer accidentally triggered from a feature branch, which deployed untested feature code to production.

Step-by-Step Resolution

Step 1 - Rollback. Deploy the last known-good image from the previous deployment. Use the deployment history to find the correct image tag.

Step 2 - Verify the damage. Check application health and data integrity. Compare behavior against the expected main branch behavior.

Step 3 - Fix the workflow. Add branch restrictions: `if: github.ref == 'refs/heads/main'` on deployment steps. Use environment protection rules that only allow main branch deployments. Add a confirmation step that displays the branch name and requires manual approval: 'You are deploying branch feature/broken-thing to production. Are you sure?'

Prevention

Always add branch validation to workflow_dispatch workflows. Use environment protection rules with branch restrictions. Add explicit confirmation steps showing what is about to be deployed. Log all manual triggers with extra detail.

Q104 GitLab CI runner token leaked - unauthorized pipelines running

What Happened

A GitLab CI runner registration token was committed to a public repository or leaked through logs. An attacker registered their own runner with this token and is executing pipelines on it - potentially exfiltrating secrets, injecting malicious code, or accessing internal networks.

Step-by-Step Response

Step 1 - Revoke the runner token immediately. In GitLab: Settings → CI/CD → Runners → Reset registration token. This invalidates all runners registered with the old token.

Step 2 - Remove unauthorized runners. Check the runner list for unknown runners. Delete any that you do not recognize.

Step 3 - Re-register legitimate runners with the new token.

Step 4 - Audit recent pipeline runs. Check for pipelines that ran on unknown runners. Review their logs for suspicious activity - data exfiltration, unauthorized deployments, secret access.

Step 5 - Rotate all secrets that were accessible to pipelines running on the compromised runner.

Prevention

Never commit runner tokens to repositories. Store tokens in secure credential management. Use short-lived runner authentication tokens (GitLab 15.x+ supports this). Monitor the runner list for unexpected additions. Alert on new runner registrations.

Q105 Pipeline deploying stale artifact - cache returning old build

What Happened

The pipeline reported a successful build, but the deployed application behaves like an older version. Investigation reveals the Docker build used cached layers from a previous build, including the application code layer, and the 'new' image is actually identical to the old one.

Why It Happened

The Dockerfile copies application code with `COPY . /app`. Docker's cache checks if the files changed by comparing checksums. If the build context was wrong (missing files, stale context), or the cache key was too broad, Docker reused the old layer containing old code.

Step-by-Step Fix

Step 1 - Force rebuild without cache. `docker build --no-cache` ensures every layer is rebuilt. Deploy the new image.

Step 2 - Find the caching issue. Check .dockerignore - is it excluding important files? Check the build context - is it the correct directory? Check the CI cache configuration - is the Docker cache being shared incorrectly between different branches?

Step 3 - Fix permanently. Use BuildKit with content-addressable cache. Ensure the build context includes all necessary files. Use --cache-from with proper cache invalidation. Add a build step that verifies the built image contains the expected code version (check a version file or git SHA baked into the image).

Prevention

Always bake the git SHA into the image as a label or file: LABEL git.sha=\$GIT_SHA. After building, verify: docker inspect image:tag shows the expected SHA. If the SHA does not match, the build used stale content.

Q106 CI/CD platform outage - can't deploy critical hotfix

What Happened

GitHub Actions (or your CI platform) is experiencing an outage. Your production application has a critical bug. The hotfix is ready but the pipeline cannot run. Users are affected.

Step-by-Step Resolution

Step 1 - Use the break-glass procedure. Every team should have a documented manual deployment procedure that bypasses CI/CD. For Kubernetes: build the image locally, push to registry, kubectl set image or helm upgrade directly. For serverless: deploy from local CLI.

Step 2 - After deploying the hotfix, create a record - a ticket, a Slack thread, an incident log. Document what was deployed, by whom, when, and why CI was bypassed.

Step 3 - When CI recovers, run the pipeline retroactively against the hotfix commit to verify it would have passed. Merge the hotfix commit to main through the normal process.

Prevention

Document a break-glass deployment procedure. Test it quarterly. Store the procedure in multiple accessible locations (wiki, runbook, printed in the operations room). Consider a secondary CI platform (self-hosted Jenkins or local scripts) for emergencies. Never let CI/CD be a single point of failure for critical deployments.

Q107 Dependency update broke production - no lockfile pinning

What Happened

The project used package.json with ^1.2.0 (caret range) and no package-lock.json committed. CI ran npm install which resolved to 1.3.0 (newly published). The 1.3.0 version had a bug. Production broke despite no code changes.

Step-by-Step Resolution

Step 1 - Identify the breaking dependency. Compare npm ls output between the working and broken builds. The newly resolved version is the culprit.

Step 2 - Pin the dependency to the last known-good version in package.json. Regenerate the lock file. Redeploy.

Step 3 - Commit the lock file. package-lock.json (npm), yarn.lock (yarn), poetry.lock (Python), go.sum (Go). Always commit lock files to Git.

Step 4 - Use npm ci instead of npm install in CI. npm ci installs exactly what the lock file specifies. npm install may update the lock file. CI should never modify the lock file - it should fail if the lock file is out of sync.

Prevention

Always commit lock files. Always use deterministic install commands (npm ci, yarn install --frozen-lockfile, pip install -r requirements.txt with pinned versions). Use Dependabot or Renovate for controlled dependency updates with testing.

Q108 Pipeline YAML merge conflict silently removed a stage

What Happened

Two developers modified the pipeline YAML on different branches. The merge conflict was resolved incorrectly - the security scanning stage was accidentally deleted. The merge passed code review because the reviewer focused on application code, not pipeline changes. Production deployments proceeded without security scanning for two weeks until someone noticed.

Step-by-Step Resolution

Step 1 - Restore the security stage. Add it back to the pipeline YAML. Verify it runs on the next pipeline execution.

Step 2 - Retroactively scan. Run security scans against all images deployed during the two-week gap. Check for vulnerabilities that should have been caught.

Step 3 - Investigate anything found. If vulnerabilities were deployed, assess impact and remediate.

Prevention

Treat pipeline YAML changes with the same rigor as infrastructure code. Require explicit review from the platform team for pipeline modifications. Add a CI check that validates the pipeline structure - verify required stages exist (build, test, security-scan, deploy). Use a pipeline linting tool or custom validation script that fails if required stages are missing.

Q109 Webhook flood - 1000 builds triggered by bot commits

What Happened

An automated bot (Dependabot, Renovate, or a custom script) created 200 PRs simultaneously, each triggering 5 pipeline jobs. 1000 jobs queued at once. Legitimate developer builds are stuck in a queue for hours. CI costs spike.

Step-by-Step Resolution

Step 1 - Cancel the queued jobs. Most CI platforms allow bulk cancellation. Cancel all bot-triggered jobs.

Step 2 - Rate-limit the bot. Configure the bot to create PRs in batches (5 at a time, not 200). Add delays between PR creations. Group minor dependency updates into a single PR.

Step 3 - Separate bot traffic. Use a different runner pool or concurrency group for bot PRs so they do not compete with developer builds. Give developer builds higher priority.

Step 4 - Add concurrency limits. Set maximum concurrent runs per workflow or per actor. GitHub Actions: concurrency group per PR. GitLab CI: resource_group.

Prevention

Configure dependency update bots with rate limits and grouping. Separate bot runner pools from developer pools. Set concurrency limits on all workflows. Monitor queue depth and alert when it exceeds normal thresholds.

Q110 Rollback deployed but old image garbage-collected from registry

What Happened

Production has a critical bug. The team attempts to rollback to the previous version. But the previous image was deleted by the registry's lifecycle policy - images older than 30 days are automatically removed. The last known-good image no longer exists.

Step-by-Step Resolution

Step 1 - Find the commit SHA. Look up the deployment history for the last good deployment's image tag (which contains the git SHA).

Step 2 - Rebuild from the exact commit. git checkout abc123 and run the build pipeline. This produces a functionally identical image (assuming reproducible builds).

Step 3 - Deploy the rebuilt image. Push to registry with the original tag. Deploy normally.

Step 4 - If the build is not reproducible (dependencies changed, base image updated), check for other copies - is the image cached on any node? Can you docker save from a running container? Check other registries or environments.

Prevention

Never garbage-collect images that are currently deployed or were recently deployed. Tag production images with a do-not-delete label. Configure lifecycle policies to skip images with production tags. Keep at least the last 5 production-deployed images regardless of age. Consider a dedicated 'golden' registry for production images with longer retention.

Interview Tips - How to Use This Guide

For Junior Roles

Focus on Q1-Q28. Know at least one tool well (GitHub Actions or GitLab CI). Understand pipeline stages, triggers, and caching. Be able to write a basic pipeline that builds, tests, and deploys a Docker image. Know the difference between CI and CD.

For Mid-Level Roles

Q1-Q62. Discuss pipeline security (OIDC!), secrets management, infrastructure pipelines, deployment strategies, and GitOps. Know how to optimize a slow pipeline. Understand monorepo CI and matrix builds. Have a real story about a pipeline issue you debugged.

For Senior Roles

All 110 questions. DORA metrics, compliance, SLSA, Tekton, progressive delivery, and migration strategies separate senior from mid-level. Be ready to discuss architecture at scale, cost optimization, and the trade-offs between all major CI/CD tools from operational experience.

For Scenario Questions

Walk through: identify symptom → check logs/metrics → hypothesize cause → verify → fix → prevent. Every answer should end with prevention - interviewers want to see you think beyond the immediate fix.

Quick Reference Cheat Sheet

Topic	Key Takeaway
CI vs CD	CI = integrate and test. CD = always deployable. Deployment = auto to prod.
Pipeline as Code	YAML/Groovy in Git. Never UI. Version, review, reproduce.
Jenkins	Controller + agents. Groovy Jenkinsfiles. Shared libraries for DRY. Plugin management is key.
GitHub Actions	YAML workflows. Marketplace actions. Pin by SHA. OIDC for cloud auth.
GitLab CI	Built-in features (registry, security, environments). DAG with needs. Merge trains.
Secrets	OIDC for cloud. Platform secrets for rest. Never in code, logs, or artifacts.
Caching	Lock file hash key. Dependencies, Docker layers, test results. 50-80% time savings.
Triggers	Fast on push. Full on PR. Deploy on merge/tag. E2E nightly. Never full on every push.
Branch protection	Required checks + reviews + no bypass. Non-negotiable.
Docker in CI	Tag with SHA. Multi-platform buildx. BuildKit cache. Never latest.
Testing	Many unit (fast), some integration (medium), few E2E (slow). Pyramid shape.
Flaky tests	Track rate. Quarantine at 1%. Delete at 5%. Never block pipeline with flakes.
Rollback	Document it. Automate it. Test it. Know it before you deploy.
Database migrations	Expand and contract. Never destructive + code change in same deploy.
GitOps	CI builds. ArgoCD deploys. CI never needs cluster credentials. Git = truth.
Approval gates	Required for prod. Required for Terraform. Never for dev/staging.
Monorepo	Detect changes. Build only affected. Nx/Turborepo/Bazel.
DORA metrics	Deploy frequency, lead time, change failure rate, MTTR. Measure and improve.

Security	Pin actions by SHA. OIDC. No fork code with secrets. Sign artifacts. SLSA.
Cost	Cache. Right-size runners. Spot instances. Skip unchanged. Fix triggers.
Jenkins migration	Templates first. Simple first. Parallel run. 3-4 months for 200 jobs.
Webhook security	Verify signatures. HTTPS. Rate limit. Rotate secrets.
Build reproducibility	Pin everything - deps, base images, tools. Lock files committed. Hermetic builds.
DAG pipelines	Use needs/DAG over linear stages. Maximize parallelism.
Deployment windows	Automate freeze enforcement. Emergency override with extra approval.